

AURONIX

Private AI Workbench

DAY 41 - AI PRODUCT BRIEF

A practical product brief about a private company AI assistant. It explains the problem, the people who would use it, the planned technical approach, how the result will be tested, and the main risk.

Core idea: let employees ask questions about private company documents, use RAG to find relevant information, and use an open-weight or locally hosted model to turn that information into a clear response.

Problem	Employees may spend too much time searching scattered internal files for one answer.
Approach	RAG retrieves useful company information and the AI model turns it into a readable response.
Privacy	The system checks access permissions before restricted information is retrieved.
Outcome	A fast internal assistant that gives source-backed answers without opening access to everything.

This is a Day 41 planning document. The purpose at this stage is to make the product idea specific enough that it can be built and evaluated in the later days.

1. The Problem I Am Trying to Solve

The main problem behind Auronix is quite practical. Companies can have a huge amount of useful information, but that information is often spread across folders, PDFs, reports, policies, and technical documents. Someone may know the answer exists somewhere and still spend a long time finding it.

There is another side to the problem. Internal documents can contain information that should stay inside the organization. Uploading those files to a public chatbot is not always a suitable option. So the product needs to make information easier to find without treating private company data as public data.

2. Required Problem Statement

Users currently search company documents and internal files manually, which can take 15-30 minutes and can lead to incomplete or inconsistent answers. An AI system could reduce that work to less than a minute with source-backed answers by finding the right authorized information and using a private AI model to explain it.

3. Who Would Actually Use Auronix?

Developers: find architecture notes, technical documentation, project details, and troubleshooting information.

Employees: ask about policies, procedures, and everyday internal processes instead of checking many files.

Managers: retrieve information from project reports and team documents more quickly.

Executives: access material relevant to their responsibilities while restricted information stays protected.

Administrators: manage users, documents, roles, and the access rules behind the knowledge base.

4. Proposed Solution - How Auronix Will Work

Auronix will use Retrieval-Augmented Generation (RAG). I am not planning to train a large language model from scratch. Instead, I will use an existing open-weight or locally hosted model and build the document, retrieval, security, and interface layers around it.

The employee experience should stay simple. The difficult work happens behind the scenes: preparing documents, finding useful information, checking the user's permissions, and sending only the approved context to the model.

STAGE	WHAT HAPPENS
1. Add documents	Bring private PDFs, DOCX files, reports, policies, and technical documents into the knowledge base.
2. Prepare content	Extract text, clean it, split it into useful sections, and attach metadata.
3. Create embeddings	Turn each section into a vector so similar ideas can be found during a search.
4. Store and index	Keep vectors and access information in a controlled vector database.
5. Ask a question	The employee asks a normal question through the Auronix chat interface.
6. Check access	The system checks the user's role before it retrieves private information.
7. Retrieve	Only relevant information that the user is allowed to see is selected.
8. Generate	The local/open-weight model writes an answer using the retrieved context.
9. Show evidence	The response points back to the source document or section used to answer the question.

5. What I Will Actually Build

The language model is only one part of the product. The main development work is the software around it.

- A web chat interface for employees.
- A backend API for users, documents, retrieval, and model requests.
- Document extraction and chunking for common file types.
- Embeddings and a vector database for semantic search.
- A RAG pipeline connecting retrieval with the language model.
- Authentication and role-based access control.
- Source display and audit logging.

6. How I Will Measure Whether It Works

I do not want to judge the product by saying the AI answers simply 'look good'. I will use a small, fixed evaluation set so that the result can be checked in a repeatable way.

MEASURE	TARGET
Answer accuracy	At least 8 out of 10 predefined company questions should be answered correctly.
Source grounding	At least 9 out of 10 answers should point to sources that really support the response.
Access control	All 10 restricted-data tests should stop an unauthorized user from receiving protected information.

7. Ten Example User Queries

These questions will become the first evaluation set and cover different kinds of requests.

1. What is Auronix mainly used for?
2. Which features are included in the current Auronix platform?
3. Can you explain the current architecture of our internal AI system?
4. Which team owns the customer-support module?
5. What should an employee do when a production incident occurs?
6. What security checks are required before an internal AI service goes live?
7. What changes are planned in the current product roadmap?
8. What are the known limitations of the Auronix assistant?
9. Give me a short summary of the latest internal project report.
10. Which parts of the executive strategy information are available to my current role?

8. Existing Tools and Where Auronix Is Different

Products such as Glean, Microsoft Copilot, and Atlassian Rovo show that enterprise search and AI are already useful together. My goal is not to copy those products. I want the Auronix prototype to explore a setup where the organization has tighter control over its own knowledge base, retrieval layer, and model hosting.

8. Existing Tools and Where Auronix Is Different

Glean: an enterprise search and AI platform that brings together information from across an organization. **My Auronix difference:** the prototype is planned around a private, company-controlled RAG pipeline and local/open-weight inference.

Microsoft Copilot: AI assistance connected to an organization's Microsoft work environment. **My Auronix difference:** the prototype is intended to be more vendor-neutral, with its own private retrieval layer and model deployment.

Atlassian Rovo: AI-powered search and knowledge features for connected workplace information. **My Auronix difference:** the prototype focuses on a customizable document-to-RAG workflow for privately managed company data.

9. Biggest Technical Risk - Accidental Information Leakage

The most serious failure would be for Auronix to retrieve information a user is not supposed to see and then include that information in the final answer. In a private-company product, that is more serious than simply returning a mediocre answer.

Mitigation plan: access rules will be attached to documents and their chunks. The user's identity and role will be checked before retrieval, so the model receives only information already filtered for that user. I will also include restricted documents in testing, keep audit records, show source references, and return a clear "not available at your access level" response when the required information is outside the user's permissions.

10. What a Successful Interaction Looks Like

Employee: "What is the process for handling a production incident?"

Auronix: returns a short answer based on the approved incident procedure and shows the source section below it. If the answer requires restricted material, the system does not reveal it.

11. Required Component List

The prototype combines normal application development with existing AI models. The aim is not to create a new LLM from scratch; it is to connect the model to private data in a useful and controlled way.

AREA	COMPONENTS NEEDED
Frontend	Chat screen, login, chat history, source display, document upload, and knowledge-base controls.
Backend	API, authentication, user management, RAG orchestration, database operations, and model communication.
Document layer	PDF/DOCX/TXT extraction, cleaning, chunking, metadata, and access labels.
AI layer	Embedding model, vector database, retriever, RAG pipeline, and open-weight/local LLM.
Security	Role-based access, document permissions, permission-aware retrieval, and audit logs.
Evaluation	Question set, accuracy checks, source checks, and restricted-information tests.

12. Final Product Goal

From the employee's point of view, Auronix should be simple: ask a question, get a useful answer, and see the evidence behind it. The complicated work - document processing, retrieval, permission checks, and model inference - should stay in the background.

The main thing I want this prototype to prove is that internal knowledge can become easier to use without making security an afterthought. The model matters, but the retrieval and access-control layers are what turn the idea into a real product.

13. Day 41 Submission Checklist

The Day 41 brief is complete when these items are covered:

- ✓ Problem statement in the required format
- ✓ Target user description
- ✓ Solution and technical approach
- ✓ Three measurable success criteria
- ✓ Ten realistic example queries
- ✓ Competitive differentiation notes
- ✓ Technical risk and mitigation plan
- ✓ Required component list

The actual coding, model integration, and testing can be completed in the following days.

14. Initial MVP Boundary

Included in the first version: private document upload, semantic search, role-aware retrieval, a simple chat interface, source references, and a small evaluation set.

Not included yet: a large enterprise integration layer, advanced analytics, fine-tuning a model from scratch, or a production-scale deployment. Those can come later once the core workflow is reliable.

15. What I Will Build First

I will first get one complete path working from start to finish: upload a small set of company documents, process them, retrieve the right passages, ask a question, generate an answer, and show the source. After that basic flow works, I can add stronger permissions, more file types, better evaluation, and a cleaner user interface.